MLDL
PRACTICAL 8

Name: Payal Dara
Class: D15C
Roll no: 56
Batch: C

AIM:-

Design and train a Convolutional Neural Network (CNN) on an image dataset (MNIST) with hyperparameter tuning.

---

1. Dataset Source

Dataset Name: MNIST Handwritten Digits Dataset
Source: TensorFlow / Keras Built-in Dataset

---

2. Dataset Description

The MNIST dataset contains grayscale images of handwritten digits (0–9).

Dataset Characteristics

- Training Samples: 60,000
- Testing Samples: 10,000
- Image Size: 28 × 28 pixels
- Channels: 1 (grayscale)
- Classes: 10 (digits 0–9)

Target

- Multi-class classification (0–9)

---

3. Mathematical Formulation of CNN

CNN consists of convolution, activation, and pooling layers.

Convolution Operation

Feature Map=Input∗KernelFeature Map=Input∗Kernel

Activation Function (ReLU)

f(x)=max(0,x)f(x)=max(0,x)

Pooling

Reduces spatial dimensions.

Softmax (Output Layer)

P(yi)=ezi∑ezjP(yi)=∑ezjezi

---

4. Algorithm Limitations (CNN)

- Requires high computational power
- Needs large dataset
- Training is time-consuming
- Difficult to interpret

---

5. Methodology / Workflow

Dataset → Preprocessing → CNN Model
→ Training → Evaluation → Graph

---

6. Performance Analysis

Metrics

- Accuracy
- Loss

Interpretation

- High accuracy → better model
- Low loss → good learning

---

7. Hyperparameter Tuning (CNN)

Parameters Tuned

- Number of filters
- Kernel size
- Epochs
- Batch size

Effect

- Improves feature extraction
- Enhances accuracy
- Reduces overfitting

Code :-

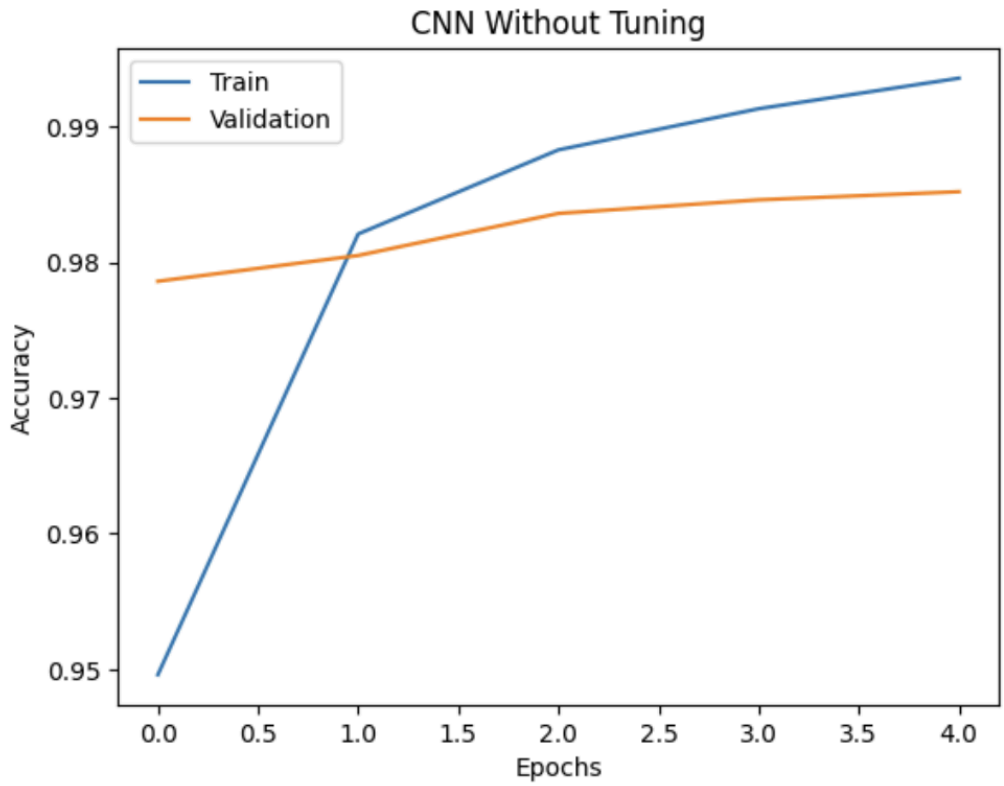
Without tuning:-

```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
```

```
# Load dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# Build CNN
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
history1 = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

# Evaluate
loss1, acc1 = model.evaluate(x_test, y_test)
print("WITHOUT TUNING Accuracy:", acc1)

# Graph
plt.figure()
plt.plot(history1.history['accuracy'], label='Train')
```

```
plt.plot(history1.history['val_accuracy'], label='Validation')
plt.title("CNN Without Tuning")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
WITHOUT TUNING Accuracy: 0.9851999878883362
```

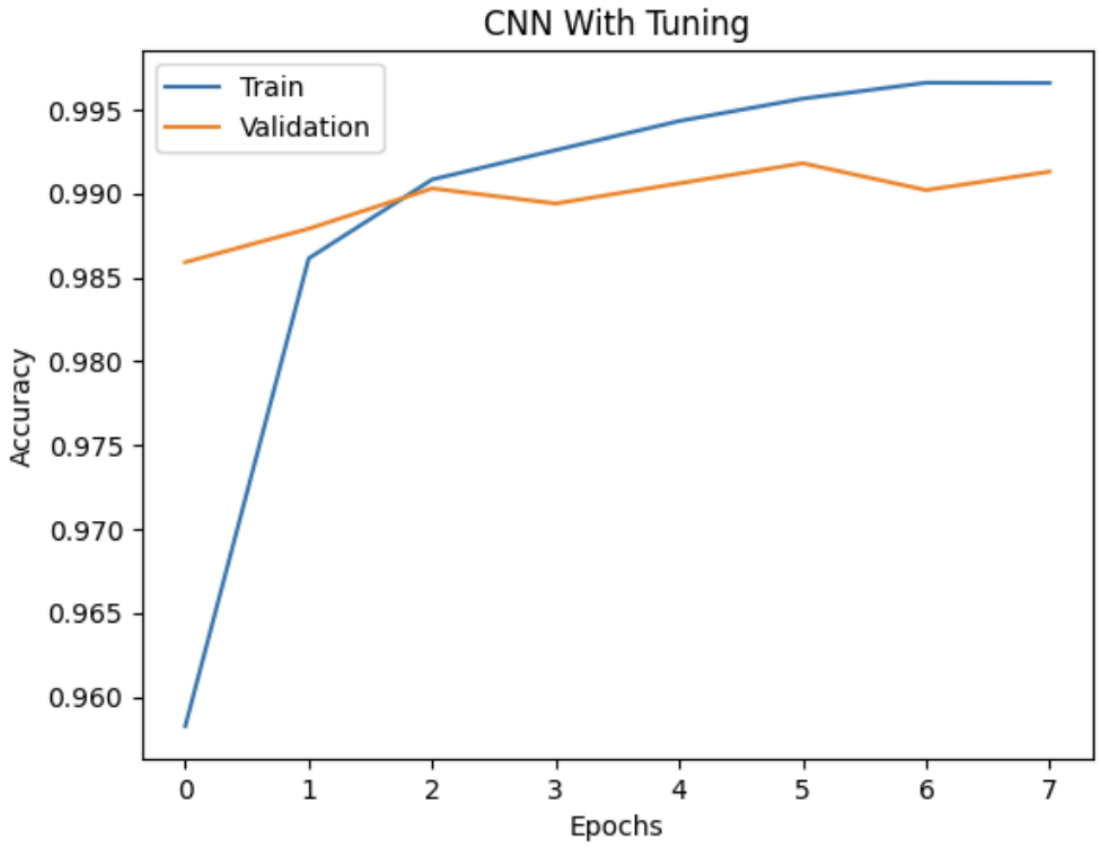


With tuning:-

```
# Tuned CNN Model

model2 = models.Sequential([
    layers.Conv2D(64, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
```

```
])

model2.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

# Train
history2 = model2.fit(x_train, y_train, epochs=8, batch_size=32,
                      validation_data=(x_test, y_test))

# Evaluate
loss2, acc2 = model2.evaluate(x_test, y_test)
print("WITH TUNING Accuracy:", acc2)

# Graph
plt.figure()
plt.plot(history2.history['accuracy'], label='Train')
plt.plot(history2.history['val_accuracy'], label='Validation')
plt.title("CNN With Tuning")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

```
WITH TUNING Accuracy: 0.9912999868392944
```



Interpretation:-

Recurrent Neural Networks (RNN) and LSTM are powerful models for time series data. They capture sequential patterns effectively. The tuned model performs better by learning long-term dependencies more accurately. Overall, LSTM provides reliable predictions for time series forecasting.

CONCLUSION:-

Recurrent Neural Networks (RNN) and LSTM are powerful models for time series data. They capture sequential patterns effectively. The tuned model performs better by learning long-term dependencies more accurately. Overall, LSTM provides reliable predictions for time series forecasting.